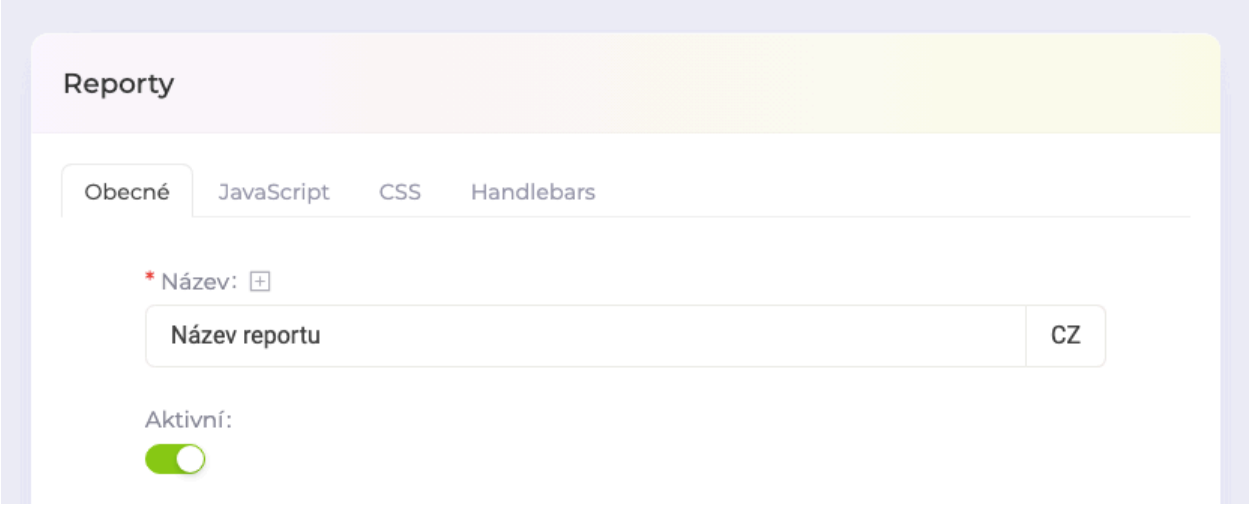
Zoty Reporty

Slouží pro hromadnou přípravu dat z databáze, možnou javascript transformaci dle požadavků uživatele a vrácení definované struktury v jednom objektu nebo JSON.
Možné použití je nejen pro samotné reporty uvnitř aplikace Zoty, ale také pro přístup externích aplikací k specifickým a naformátovaným datům.

URL

Hlavní menu > Administrace > Nastavení > Reporty



Záložky

Obecné

Atributy a vlastnosti reportu. Pouze reporty označené jako “aktivní” jsou viditelné v Zoty.
U reportu je možné nastavit jeho omezení jen pro konkrétní uživatele, nebo dle role.
Dále je možné omezit jeho zobrazení pro konkrétní sekci (pohled) jako Úkoly, Rizika a jiné.
Případně pro jakou Kategorii, nebo Typ entity se může použít.

JavaScript

Slouží k získání, přípravě dat pro Report. JS data v reportu **je nutné vrátit v proměnné** `result`, která slouží jako “obálka” pro data. Obsah této proměnné je předán do Handlebars template (HBS) a její samotný název zaniká.

CSS

Report vždy obsahuje základní nastavení stylů, které lze nahradit, rozšířit, nebo upravit o další CSS vlastnosti.

Handlebars

K prezentaci dat slouží HBS template https://handlebarsjs.com/guide/, ve kterém je možné využít pomocné “Helpery” (getTranslation, getValueByKey a jiné).

Ukázky použití

Záložka JavaScript

```
18
19  const costsEntityList = (entityList || []).map(entity => ({
20    id: entity.id,
21    costsCapex: entity.costsCapex,
22    costsOpex: entity.costsOpex,
23  }));
24
25  const result = {
26    entityMap: costsEntityList.reduce((map, item) => {
27      map[item.id] = item;
28      return map;
    }, {}),
  };
```

```
// Příklad #1

const entityList = getGraphql({
  query: `query getEntityListCustomQuery($input: ConnectionInputType) {
    entityListCustomQuery(input: $input) {
      id, entityTypeId, entityCategory, attributeList, attributeSystemList
    }
  }`,
  variables: {
    input: {
      fieldList: ['id', 'entityTypeId', 'entityCategory', 'attribute.name',
'attribute.costs'],
      filter: {
        id: {{dataIdList}},
      },
    },
  },
});
```

```
const costsEntityList = (entityList || []).map(entity => ({
  id: entity.id,
  nameTranslated: entity.nameTranslated,
  costsCapex: entity.costsCapex,
  costsOpex: entity.costsOpex,
}));

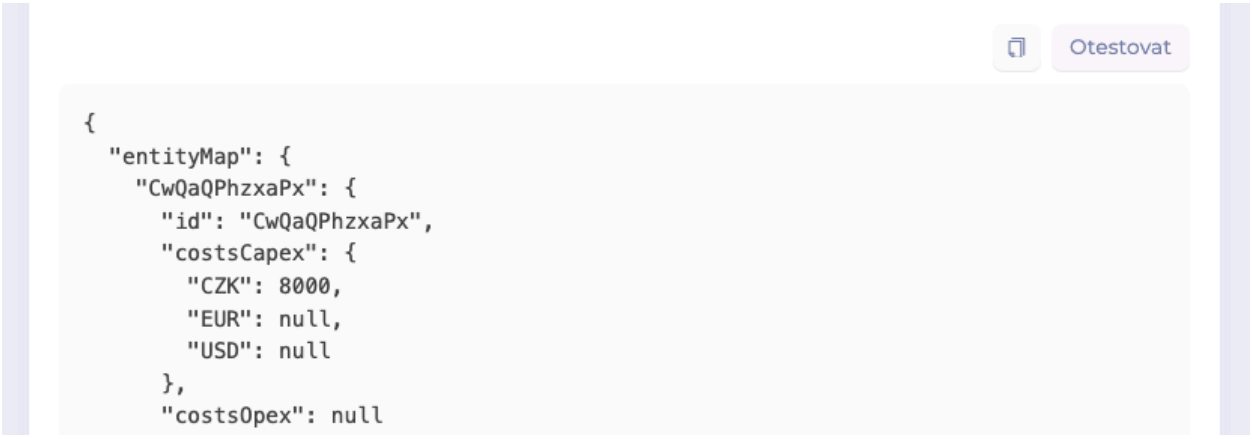
const result = {
  entityMap: costsEntityList.reduce((map, item) => {
    map[item.id] = item;
    return map;
  }, {}),
};
```

V Handlebars template (HBS) je k dispozici objekt s klíčem `entityMap` (vrchní uzel `result` je ignorován), který obsahuje objekt/mapu entit s ID entity jako `key`. V HBS se s ní může pracovat jako s Object, tak i jako Array. Pokud je proměnná pole, chová se jako Array.

```
{{#each entityMap}}
  {{ @key }}: {{ this.costsCapex.CZK }}
{{/each}}
```

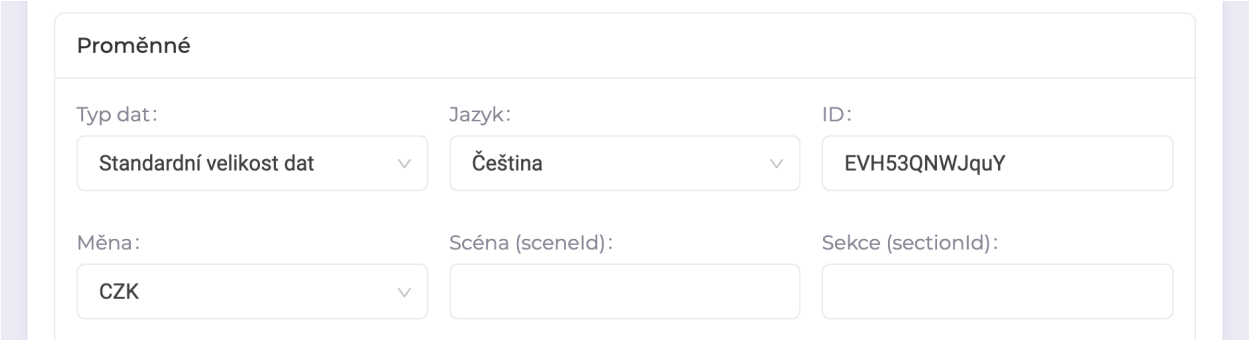
Test kódu

Pod JavaScript tabem je možné kód otestovat a získat představu o podobě dat. Zobrazená data jsou v této podobě předána do HBS template.



Proměnné

Pro specifické otestování uživatelem je možné využít volitelné proměnné. Při reálném použití jsou automaticky vyplněna na základě aktuální scény/sekce, nebo vybraných datech.



ID (entit/y, nebo auditu,...) je možné zadat jednotlivě “CwQaQPhzxaPx” nebo více ID zadáme oddělené čárkou “CwQaQPhzxaPx, Cx5CYnrdTd26”. V JavaScript kódu využijeme string (bez uvozovek) `{{dataId}}`, nebo `{{dataIdList}}` v případě zadání více ID (Příklad #1). Standardně bude tento string automaticky vyplněn dle contextu spuštění reportu.

CSS

```css
.table-gray tr:nth-child(even) td {
  width: 830px;
  background: #aaa;
  color: #fff;
}
```

Handlebars

```
<html>
<head>
  <title>Seznam rocnich nakladu</title>
</head>
<body>
  <table class="table-data table-gray">
    <thead>
    <tr>
      <th>{{ getTranslation 'entity.attribute.key' 'Key' }}</th>
      <th>{{ getTranslation 'entity.attribute.name' 'Name' }}</th>
      <th>{{ getValueByKey (getEntityAttribute 'costs') 'nameTranslated' }}
CAPEX </th>
      <th>{{ getValueByKey (getEntityAttribute 'costs') 'nameTranslated' }}
```

```
OPEX</th>
    </tr>
    </thead>
    <tbody>
    {{#each entityMap }}
      <tr>
        <td class="u_noWrap">
          {{ key }}
        </td>
        <td class="nameWidth">
          {{ nameTranslated }}
        </td>
        <td class="u_noWrap">
          {{ getValueByKey costsCapex ../_currency }}
        </td>
        <td class="u_noWrap">
          {{ getValueByKey costsOpex ../_currency }}
        </td>
      </tr>
    {{/each}}
    </tbody>
  </table>
</body>
</html>
```

Výsledkem je PDF generované z HTML, které je stylově upravené za pomoci CSS. Data z JavaScript tabu jsou zobrazena za pomoci HBS.

Klíč	Název	Náklady CAPEX	Náklady OPEX
TASK7	Ukol 2b		
TASK6	Ukol 4	4000	
TASK5	Ukol 3		
TASK4	Ukol 2a	8000	
TASK3	Ukol 1	5000	4000

JS

Systémové funkce

getGraphql

Nejčastěji využívanou systémovou funkcí je `getGraphql`, která slouží pro získání většiny dat z databáze za pomoci GraphQL. Musí obsahovat `query` [String] a "volitený" klíč `variables`, který je zpravidla **doporučován** nejen pro filtrování dat `filter`.

```
const riskEntityList = getGraphql({
  query: `query getEntityListCustomQuery($input: ConnectionInputType) {
    entityListCustomQuery(input: $input) {
      id, key, attributeList
    }
  }`,
  variables: {
    input: {
      fieldList: ['id', 'key', 'attribute.name'],
      filter: {
        entityTypeId: 'risk',
      },
    },
  },
});
```

getUserListRequest

Funkce pro získání uživatelů účtu přihlášeného uživatele.

```
const userList = getUserListRequest();

const filteredUserList = getUserListRequest({
  filter: {
    username: ['username1', 'username2'],
  },
});
```

Ostatní funkce

GraphQL by mělo být primárním zdrojem dat z databáze. Reporty ale umožňují i práci s jednotlivými modely databáze.

`getEntityListRequest` (Entity),
`getAssignmentListRequest` (Připojení uživatelé k entitám),
`getAttributeListRequest` (Atributy entit),
`getAuditListRequest` (Audity),
`getCommentListRequest` (Komentáře),
`getDataSetListRequest` (Datové sady jako RTP a jiné plány),
`getEntityTypeListRequest` (Typy jednotlivých entit),
`getEvaluationListRequest` (Ohodnocení dle schémat),
`getExternalSourceListRequest` (Externí data, ARIS),
`getFileListRequest` (Připojené soubory),
`getFragmentListRequest` (Fragmenty typu paragraf zákona,...),
`getGroupHasAuditListRequest` (Audity připojené do skupin),
`getGroupHasDataSetListRequest` (Datové sady připojené do skupin),
`getGroupHasEntityListRequest` (Entity připojené do skupin),
`getGroupHasFragmentListRequest` (Fragmenty připojené do skupin),
`getGroupHasUserListRequest` (Uživatelé připojené do skupin),
`getGroupListRequest` (Skupiny a Štítky),
`getMeasureListRequest` (Vytvořená Opatření),
`getPeopleListRequest` (Seznam lidí, kteří nejsou uživatelé Zoty),
`getRelationListRequest` (Relace/Propojení mezi entitami),
`getStateListRequest` (Možné stavy entit),
`getSurveyResultListRequest` (Výsledky dotazníků).

```
const entityList = getEntityListRequest({
  filter: {
    entityCategory: 'assets',
  },
});
```

i18n

Funkce pro překlady. První argument je klíč překladu, druhým volitelné parametry pro dynamický text.

```
const msg = i18n('entity.messages.createdByUser', { userLink: "Test" });
// msg = "Vytvořeno uživatelem Test."
```

getZotyHelpers.date

K manipulaci s datumem je možné využít helper “date”. Prvním argumentem je jméno helperu a následuje datum, které chceme upravovat. Pro aktuální čas je možné jako druhý argument zvolit prázdný string. Třetím argumentem jsou manipulační funkce.

```
// now = /aktualni datum/
const now = getZotyHelpers('date', null);

// mesicPoLednu = "2023-02-01"
const mesicPoLednu = getZotyHelpers('date', '2023-01-01', 'add(1,
"month")', 'format(YYYY-MM-DD)');
```

getZotyHelpers.request

Pro práci s daty z externích zdrojů je možné využít request. Standardně jsou podporovány typy `get`, `post`, s URL adresou. Druhým argumentem po `'request'` je objekt (nejedná se o string) dle Axios Options.

```
const requestGet = getZotyHelpers('request', { method: "get",
url:"https://httpbin.org/get" });
```

Grafy

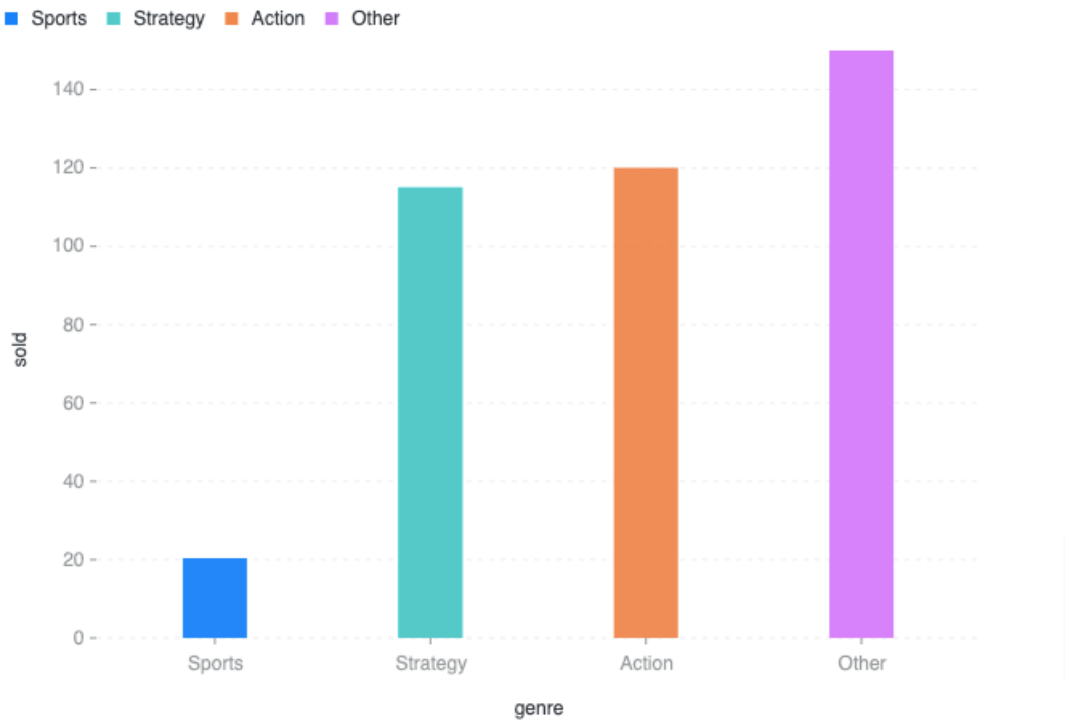
Grafy lze generovat za pomoci pole `chartList` v první úrovni `result`. Mimo definici šířky a výšky (není povinné, default 640x480), obsahuje `config` samotné `options`, který obsahuje data a lze jím nastavit podobu grafu dle https://g2.antv.antgroup.com/en/examples/.

Důležité je unikátní ID `componentId`, které lze na jakémkoliv místě v HBS zobrazit za pomoci {{{ chart1 }}} (tři závorky jsou povinné).

```
chartList: [{
    componentId: 'chart1',
    config: {
      width: 640,
      height: 480,
      options: {
        type: "interval",
        data: [
```

```
        { genre: "Sports", sold: 0 },
        { genre: "Strategy", sold: 115 },
        { genre: "Action", sold: 120 },
        { genre: "Other", sold: 150 },
      ],
      encode: { x: "genre", y: "sold", color: "genre", size: 40 },
    },
  },
  title: 'Muj graf #1',
}],
```

Výsledkem kódu výše je graf



Handlebars

Systémové proměnné

_locale

Vybraný kód jazyka [CZ, EN, DE]

_localeDate

Dle kódu jazyka se automaticky vybere lokalizace pro datumové atributy [cs, en, de]

_currency

Vybraná měna [CZK, USD, EUR]

Zoty helpers

Pro jednodušší generování základních komponent je možné využít pomocné HBS partials.

getTranslation

Obdobně jako i18n v Javascript můžeme využít helper getTranslation pro získání základních překladů `{{ getTranslation 'entity.attribute.key' 'Key' }}`

Otázky a odpovědi

Co znamená chyba “TypeError: # could not be cloned.”, při testu JS kódu?
Pravděpodobně jste někde opomenuli “await” před asynchronní funkcí (await getGraphql,…).

Příklady

Úprava rolí uživatele

```
const updatedUser = await getGraphql({
  query: `mutation updateUser($input: UserUpdateInputType!) {
updateUserMutation(input: $input) { id roleList }}
`,
  variables: {
    input: {
      id: 'ABCDedZGXd4a',
      roleList: ['user'],
    },
  },
});
```